Create and Connect Rows

We will now **create rows** for each table. ✨

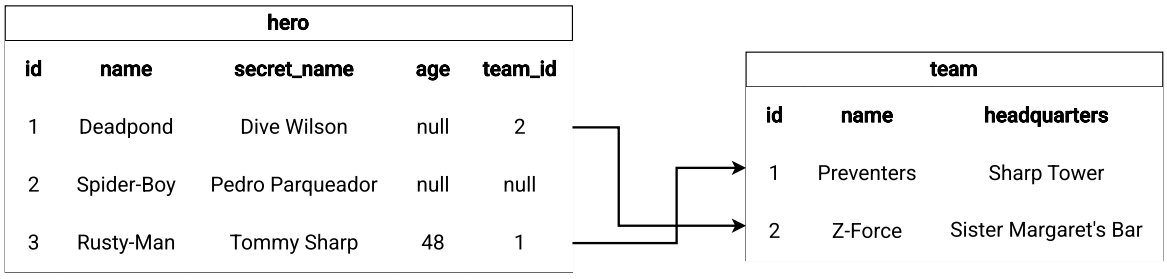
The `team` table will look like this:

id	name	headquarters
1	Preventers	Sharp Tower
2	Z-Force	Sister Margaret's Bar

And after we finish working with the data in this chapter, the `hero` table will look like this:

id	name	secret_name	age	team_id
1	Deadpond	Dive Wilson	null	2
2	Rusty-Man	Tommy Sharp	48	1
3	Spider-Boy	Pedro Parqueador	null	null

Each row in the table `hero` will point to a row in the table `team`:



> **Info**
>
> We will later update **Spider-Boy** to add him to the **Preventers** team too, but not yet.

We will continue with the code in the previous example and we will add more things to it.

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def main():
    create_db_and_tables()


if __name__ == "__main__":
    main()
```

Make sure you remove the `database.db` file before running the examples to get the same results.

Create Rows for Teams with SQLModel

Let's do the same we did before and define a `create_heroes()` function where we create our heroes.

And now we will also create the teams there. 🎉

Let's start by creating two teams:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

This would hopefully look already familiar.

We start a **session** in a `with` block using the same **engine** we created above.

Then we create two instances of the model class (in this case `Team`).

Next we add those objects to the **session**.

And finally we **commit** the session to save the changes to the database.

Add It to Main

Let's not forget to add this function `create_heroes()` to the `main()` function so that we run it when calling the program from the command line:

Python 3.10+

```python
# Code above omitted 👆

def main():
    create_db_and_tables()
    create_heroes()

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Run it

If we run that code we have up to now, it will output:

```bash
```

You can see in the output that it uses common SQL `INSERT` statements to create the rows.

Create Rows for Heroes in Code

Now let's create one hero object to start.

As the `Hero` class model now has a field (column, attribute) `team_id`, we can set it by using the ID field from the `Team` objects we just created before:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

We haven't committed this hero to the database yet, but there are already a couple of things to pay **attention** to.

If the database already had some teams, we wouldn't even know **what is the ID** that is going to be automatically assigned to each team by the database, for example, we couldn't just guess `1` or `2`.

But once the team is created and committed to the database, we can access the object's `id` field to get that ID.

Accessing an attribute in a model that was just committed, for example with `team_z_force.id`, automatically **triggers a refresh** of the data from the DB in the object, and then exposes the value for that field.

So, even though we are not committing this hero yet, just because we are using `team_z_force.id`, that will trigger some SQL sent to the database to fetch the data for this team.

That line alone would generate an output of:

```
INFO Engine BEGIN (implicit)
INFO Engine SELECT team.id AS team_id, team.name AS team_name, team.headquarters AS
team_headquarters
FROM team
WHERE team.id = ?
INFO Engine [generated in 0.00025s] (2,)
```

Let's now create two more heroes:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

When creating `hero_rusty_man`, we are accessing `team_preventers.id`, so that will also trigger a refresh of its data, generating an output of:

```
INFO Engine SELECT team.id AS team_id, team.name AS team_name, team.headquarters AS
team_headquarters
FROM team
WHERE team.id = ?
INFO Engine [cached since 0.001795s ago] (1,)
```

There's something else to note. We marked `team_id` as `int | None`, meaning that this could be `NULL` on the database (and `None` in Python).

That means that a hero doesn't have to have a team. And in this case, **Spider-Boy** doesn't have one.

Next we just commit the changes to save them to the database, and that will generate the output:

```
INFO Engine INSERT INTO hero (name, secret_name, age, team_id) VALUES (?, ?, ?, ?)
INFO Engine [generated in 0.00022s] ('Deadpond', 'Dive Wilson', None, 2)
INFO Engine INSERT INTO hero (name, secret_name, age, team_id) VALUES (?, ?, ?, ?)
INFO Engine [cached since 0.0007987s ago] ('Rusty-Man', 'Tommy Sharp', 48, 1)
INFO Engine INSERT INTO hero (name, secret_name, age, team_id) VALUES (?, ?, ?, ?)
INFO Engine [cached since 0.001095s ago] ('Spider-Boy', 'Pedro Parqueador', None, None)
INFO Engine COMMIT
```

Refresh and Print Heroes

Now let's refresh and print those new heroes to see their new ID pointing to their teams:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

If we execute that in the command line, it will output:

```bash
```

They now have their `team_id`s, nice!

Relationships

Relationships in SQL databases are just made by having **columns in one table** referencing the values in **columns on other tables**.

And here we have treated them just like that, more **column fields**, which is what they actually are behind the scenes in the SQL database.

But later in this tutorial, in the next group of chapters, you will learn about **Relationship Attributes** to make it all a lot easier to work with in code. ✨